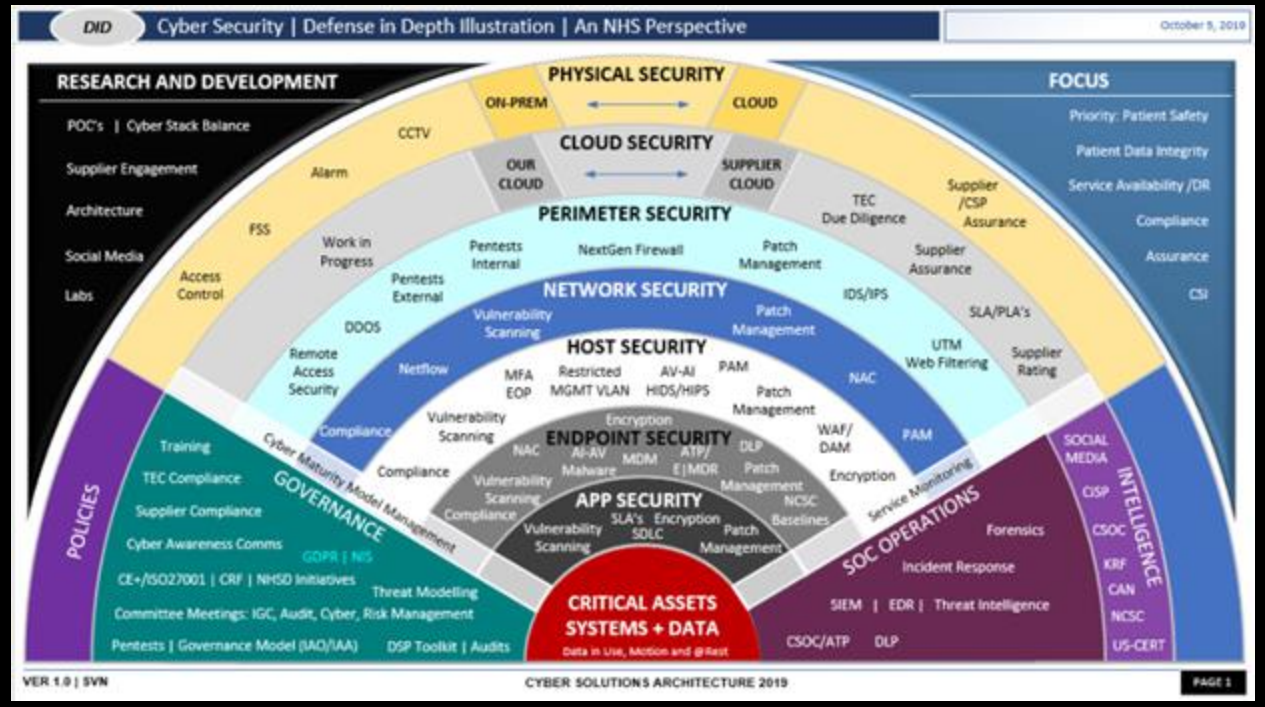
Green Pace

Security Policy Presentation

Developer: *Gary Travis*



OVERVIEW: DEFENSE IN DEPTH



- Green Pace builds custom applications that handle customer, financial, and internal data, so a consistent secure development policy is required.
- The policy defines ten core security principles, C/C++ coding standards, encryption rules, and Triple-A (authentication, authorization, accounting) practices.
- Together, these controls create defense in depth: multiple layers of protection in code, configuration, and operations so that one failure does not lead directly to compromise.

Green Pace

THREATS MATRIX

• STD-002-CPP: Validate data values and ranges • STD-004-CPP: SQL Injection • STD-010-CPP: Validate all external input at API/UI boundaries	• STD-003-CPP: Bounded strings / buffer overflows • STD-005-CPP: RAII and smart pointers (memory safety) • STD-007-CPP: Exceptions and strong exception safety • STD-008-CPP: Secure logging • STD-009-CPP: Generic error messages
• STD-006-CPP: Assertions used correctly (medium severity, lower priority)	

Green Pace

10 PRINCIPLES

- Validate Input Data
- Heed Compiler Warnings
- Architect and Design for Security Policies
- Keep It Simple
- Default Deny
- Least Privilege
- Sanitize Data Sent to Other Systems
- Defense in Depth
- Use Effective Quality Assurance Techniques
- Adopt a Secure Coding Standard

Principle 1: STD-001, STD-002, STD-010
Principle 3: STD-006, STD-007, STD-009
Principle 7: STD-004, STD-009, STD-008
Principle 8: STD-002, STD-004, STD-005, STD-007, STD-008, STD-010
Principle 10: All ten coding standards

Green Pace

CODING STANDARDS

- "Our ten C/C++ coding standards cover: data types, value ranges, string handling, SQL injection, memory protection (RAII), assertions, exceptions, secure logging, error messages, and boundary input validation at APIs and UIs."
- "We prioritized standards by Severity, Likelihood, and Level from the risk tables: Critical issues with high likelihood and low remediation cost (such as injection and unvalidated input) are addressed first."
- "Top priorities: input validation (STD-002, STD-010), SQL injection (STD-004), and buffer and string safety (STD-003); these are most likely to lead to data theft or code execution."

Green Pace

ENCRYPTION POLICIES

- Encryption at rest: “All customer and sensitive internal data stored on disks, databases, and backups must be encrypted with strong algorithms and centrally managed keys, so stolen media cannot be read.”
- Encryption in flight: “All traffic carrying sensitive or authenticated data must use TLS (HTTPS, secure DB connections) to prevent eavesdropping or tampering on the network.”
- Encryption in use: “Applications must minimize time that sensitive data is kept in memory, avoid writing secrets to logs or temporary files, and restrict access to processes that need them.”

Green Pace

TRIPLE-A POLICIES

- Authentication: “Unique accounts, secure logins, and where possible multi-factor authentication for all systems; all login attempts are logged for auditing.”
- Authorization: “Role-based access control with least privilege; any change to user permissions or roles must be requested, approved, and logged.”
- Accounting: “Log key events such as logins, file access, database changes, and user management actions; protect logs from tampering and retain them for audits and incident response.”

Green Pace

Unit Testing

Unit Testing: Input Validation for API/UI (STD-010-CPP)

Vulnerability tested: using unvalidated external input for counts, allocations, or array indexes.

Target functions: parse_positive_int and handle_request_compliant.

Goal: prove that valid inputs are accepted and unsafe inputs are rejected without creating arrays or writing out of bounds.

Tools: Visual Studio C++ unit test framework plus static analysis (Cppcheck, SonarQube) to confirm checks are present in code.

Green Pace

Does valid numeric input create the correct number of items?

- On the slide, show:
- Short description:
  - Input: `"10"` passed to `handle_request_compliant`.
  - Expected: `parse_positive_int` returns true, count = 10, items array of size 10 allocated and filled.
- What the unit test does (bullets):
  - Calls `handle_request_compliant("10")`.
  - Asserts that no error message is printed.
  - Optionally checks that the function completes without throwing and that the expected side effect occurs (for example, logging or a return code).
- Result text:
  - "Result: Test passes. Valid numeric input is accepted and processed safely."

Does non-numeric input get rejected without allocation?

- Description:
  - Input: `"abc"` passed to `handle_request_compliant`.
  - Expected: `parse_positive_int` returns false and `handle_request_compliant` prints `"Invalid item count."` and returns without allocating items.
- What the test does:
  - Calls `handle_request_compliant("abc")`.
  - Captures console output in the test.
  - Asserts that the output contains `"Invalid item count."`.
  - Confirms that no allocation or loop over items occurs (for example, by checking a return code or an observable side effect).
- Result:
  - “Result: Test passes. Non-numeric input is rejected and no unsafe memory operations occur.”

Are zero or negative counts rejected safely?

- Description:
  - Inputs: `"0"` and `"-5"` passed separately.
  - Expected: `parse_positive_int` either fails or returns a value that fails the check `count <= 0 || count > MAX_ITEMS`, causing `"Invalid item count."` to be printed.
- What the test does:
  - Calls `handle_request_compliant("0")` and `handle_request_compliant("-5")`.
  - Captures console output.
  - Asserts that both calls produce `"Invalid item count."` and return early.
- Result:
  - “Result: Test passes. Zero and negative values never reach the allocation, preventing invalid sizes and negative indexing.”

Does an excessively large number get blocked before overflow?

- Description:
  - Input: a very large string such as `"9999999999"` or any value larger than `MAX_ITEMS` and possibly larger than `int` can hold.
  - Expected: `parse_positive_int` detects overflow or returns a value that exceeds `MAX_ITEMS`, and the function prints `"Invalid item count."`.
- What the test does:
  - Calls `handle_request_compliant("9999999999")`.
  - Captures console output and asserts the error message appears.
  - Optionally runs with AddressSanitizer in debug builds to confirm no overflow or invalid allocation occurs.
- Result:
  - “Result: Test passes. Overly large counts are rejected, preventing integer overflow and oversized allocations.”

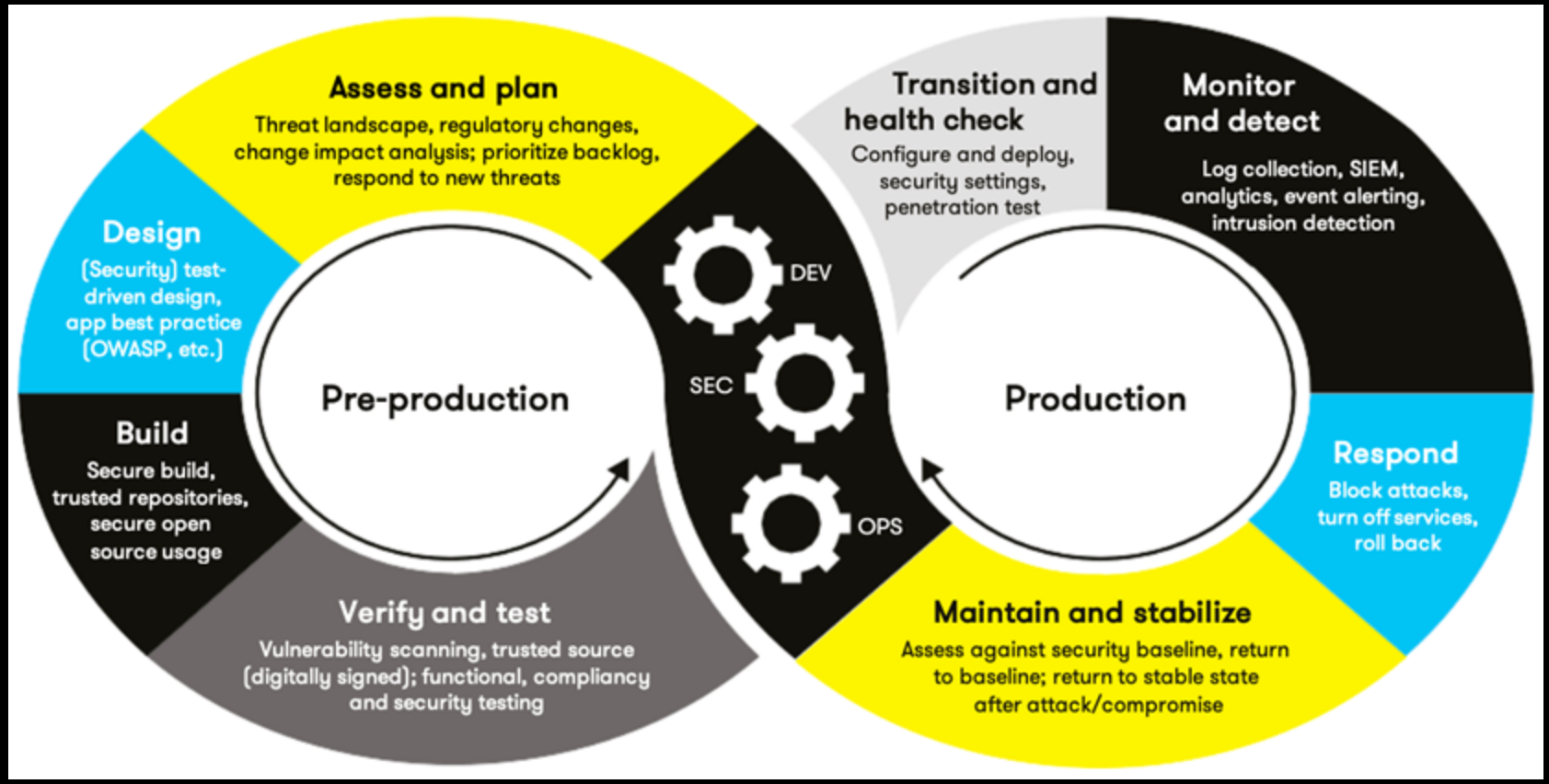
Does the maximum allowed value still work correctly?

- Description:
  - Input: `"1000"` when `MAX_ITEMS = 1000`.
  - Expected: `parse_positive_int` succeeds, count = 1000, items array of size 1000 allocated, loop writes exactly 1000 elements and then deletes the array safely.
- What the test does:
  - Calls `handle_request_compliant("1000")`.
  - Asserts that no error message appears.
  - Confirms that the function returns successfully without memory errors (checked via tests and, optionally, ASan).
- Result:
  - “Result: Test passes. The upper boundary is handled correctly, which shows that the range checks are implemented properly.”

Do values with leading zeros still parse correctly

- Description:
  - Input: `"007"` (still a valid positive number).
  - Expected: `parse_positive_int` returns true, count = 7, items are allocated and filled correctly.
- What the test does:
  - Calls `handle_request_compliant("007")`.
  - Asserts that no error message is shown and that the function completes normally.
- Result:
  - "Result: Test passes. Inputs with leading zeros are accepted as valid numbers, while still rejecting non-numeric and out-of-range values."

AUTOMATION SUMMARY



Green Pace

TOOLS

- The DevSecOps pipeline at Green Pace uses stages of Plan/Code, Build/Test, Release/Deploy, and Operate/Monitor, and each stage includes specific security activities tied to this policy.
- Static analyzers such as Cppcheck and SonarQube run in the Code and Build/Test stages to detect missing input validation, unsafe memory usage, SQL injection patterns, and logging of sensitive data.
- The Visual Studio C++ unit test framework and runtime tools like AddressSanitizer are used in Test to prove that functions like `parse_positive_int` and secure logging behave correctly with both valid and invalid inputs.
- Log management platforms, along with existing OS, firewall, and anti-malware logs, are used in Operate/Monitor to detect suspicious behavior and verify that authentication, authorization, and encryption policies are followed.

Green Pace

RISKS AND BENEFITS

- Problems: Legacy code and inconsistent practices can leave input validation, memory management, and error handling incomplete, which increases the chance of critical vulnerabilities reaching production.
- Solutions: Apply the ten C/C++ coding standards, enforce encryption for data at rest and in flight, and follow the Triple-A policies for controlling and auditing access.
- Risks and benefits if we act now: Refactoring and adding tests requires time and training, but it reduces the likelihood of data breaches, service outages, and expensive incident response.
- Risks if we wait: Unfixed vulnerabilities remain in the code, giving attackers a larger surface area and increasing both technical risk and compliance risk.
- Recommended order: First address Critical and High issues such as SQL injection, boundary input validation, and buffer handling, then handle medium-severity items such as assertion usage and exception patterns.

Green Pace

RECOMMENDATIONS

- Extend the policy to cover third-party libraries and dependencies, including regular updates and vulnerability scanning, so external components do not introduce new risks.
- Strengthen code review by adding checklists aligned with the ten principles and coding standards, ensuring reviewers consistently check for security issues.
- Increase automated test coverage around high-risk workflows such as authentication, authorization changes, and access to sensitive data.
- Regularly update the threats matrix and risk levels based on new incidents, tool findings, and industry guidance to keep priorities aligned with real-world threats.

Green Pace

CONCLUSIONS

- Green Pace now has a standardized secure coding policy that connects ten guiding principles to ten C/C++ coding standards, encryption policies, and Triple-A practices.
- Embedding external tools and automated tests into the DevSecOps pipeline allows the team to detect and prevent common vulnerabilities before deployment.
- Making these standards mandatory for new and changed code and continuously improving tools and tests will help maintain a strong security posture over time.



REFERENCES

- KirkpatrickPrice. (2023, December 26). *8 best secure coding practices*. KirkpatrickPrice. https://kirkpatrickprice.com/blog/secure-coding-best-practices/
- Oligo Security. (2025). *Secure coding: Top 7 best practices, risks & future trends*. Oligo Security Academy. https://www.oligo.security/academy/secure-coding-top-7-best-practices-risks-and-future-trends
- Microsoft. (n.d.). *Unit testing C/C++ code in Visual Studio*. Microsoft Learn. (Add the full URL you actually used.)

Green Pace